A03a · The Function That Calls Itself — Practice A2 · 9618 §19.2

Name: _____ Due: _____

This topic is examined **twice** — described and traced on Paper 3, typed on Paper 4, so this homework has both. **Questions 1–6 are written, on this sheet. Questions 7–10 are code, in `A03a_Practice.java`** (listed at the bottom). One set of numbers across both halves.

1. Say it for marks. State **three essential features of recursion**. [3]

1

2

3

2. Trace it. Complete the trace table for the function below when it is called as `Total(7)`. [5]

```
FUNCTION Total(N : INTEGER) RETURNS INTEGER
   IF N <= 0 THEN
      Result ← 0
   ELSE
      Result ← N + Total(N - 2)
   ENDIF
   RETURN Result
ENDFUNCTION
```

Call number	Function call	N	Result	Return value

Use the exam's convention: a call number in **brackets** means you are back inside that call.

3. The one that gets away. A student writes a recursive function whose general case is `RETURN Countdown(N - 3)` and whose base case is `IF N = 0 THEN RETURN 0`. They call it with `Countdown(10)` and the program crashes. [3]

(a) Explain why it crashes, even though the calls are moving *towards* the base case. [2]

(b) State a one-word change to the base case that fixes it for every starting value. [1]

4. Read it. State what this function calculates, and give the value of `Mystery(7, 3)`. [2]

```
FUNCTION Mystery(A : INTEGER, B : INTEGER) RETURNS INTEGER
   IF B = 0 THEN
      RETURN 0
   ELSE
      RETURN A + Mystery(A, B - 1)
   ENDIF
ENDFUNCTION
```

calculates

`Mystery(7, 3)` =

5. Walk the tree. An ordered binary tree is stored in the 2D array `ArrayNodes` below: column 0 is the left pointer, 1 the data, 2 the right pointer, and `−1` is null. `RootPointer = 0`. [4]

Index	LeftPtr	Data	RightPtr
0	1	40	2
1	3	23	4
2	−1	55	5
3	−1	8	−1
4	−1	31	−1
5	−1	60	−1

> A **post-order** traversal visits the **left** subtree, then the **right** subtree, then outputs the node.

(a) Write the output of a post-order traversal starting at the root. [2]

(b) State the **height** of this tree (the number of levels), and the number of **leaf** nodes. [2]

height

leaves

6. Choose. For each task, state whether recursion or iteration is the better choice, and give **one** reason. [2]

(a) Adding up every number in a 1D array of 1000 elements.

(b) Printing every node of a binary tree in order.

💻 **QUESTIONS 7–10 — ON THE COMPUTER, NOT ON THIS SHEET.** Open **`A03a_Practice.java`**. It contains a short unmarked warm-up and then these questions. The file **tests itself** — including against the exact ten values Cambridge used in the November 2021 paper, and it **reads your source** to check you solved these by recursion and not with a loop.

7	**`PostOrder()`** — left, right, then output the node; recursive	[7]
8	**`CountNodes()`** — how many nodes are in the tree, without a counter variable	[3]
9	**`TreeHeight()`** — how many levels; an empty tree is 0	[3]
10	🍦 **`PreOrder()`** — the third traversal — optional, not marked	—

To hand in: rename the file to **`A03a_YourName.java`** and submit it. Change nothing inside — the class is not **`public`**, so Java does not mind what the file is called. · **Written 19 marks + code 13 marks = 32.**

🕳 **Want to dig?** Question 2's function adds 7 + 5 + 3 + 1. Try it for **`Total(9)`**, **`Total(11)`**, **`Total(13)`** and look at the answers next to the number of terms you added. The pattern was known to the Pythagoreans about 2 500 years ago, and it has a one-line picture proof — draw the odd numbers as L-shapes nested around each other and see what they build.

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.